

Клон украл мой меч

Ссылки, изменяемость, копирование | Слайды лекции: 4-8, 22

В игре появился дешёвый сервис клонирования. Клон получил ваш меч, а затем положил в рюкзак утку. Утка почему-то оказалась и у вас. С монетами такой фокус не сработал.

Исходный фрагмент

```
bag = ["меч"]
clone = bag
coins = 10
clone_coins = coins
clone.append("утка")
clone_coins += 5
print(bag)
print(bag is clone)
print(coins, clone_coins)
```

Что сделать

1. До запуска запишите три строки вывода и объясните каждую.
2. Напишите `clone_inventory(items: list[str]) -> list[str]`: новый список с теми же предметами в том же порядке.
3. Изменение нового списка не должно менять исходный. Все предметы здесь строки; вложенных контейнеров нет.

Примеры и ожидаемое поведение

```
bag = ["меч"]
copy = clone_inventory(bag)
copy.append("утка")
assert bag == ["меч"]
assert copy == ["меч", "утка"]
assert copy is not bag
```

Проверьте ещё. Пустой рюкзак и два последовательных вызова функции.

Объясните. Что здесь меняет объект, а что меняет привязку имени? Почему достаточно поверхностной копии?

Босс с нулём здоровья

None, bool, ложные значения | Слайды лекции: 14-15, 23

Гоблин выключил дружественный огонь, убрал никнейм и честно поставил себе 0 HP. Загрузчик настроек решил, что всё это пустяки, и воскресил гоблина с настройками по умолчанию.

Исходный фрагмент

```
# Так загружается здоровье в сломанной версии:  
hp = config.get("hp") or 100
```

Что сделать

1. Напишите `boss_settings(config: dict) -> dict`. Верните новый словарь ровно с ключами `hp`, `nickname`, `friendly_fire`.
2. Значения по умолчанию: 100, строка Гоблин и `True` соответственно. Подставляйте их только при отсутствующем ключе или значении `None`.
3. Значения 0, пустая строка и `False` нужно сохранить. Лишние ключи проигнорируйте; исходный словарь не изменяйте. Другие значения по типу проверять не требуется.

Примеры и ожидаемое поведение

```
boss_settings({"hp": 0, "nickname": "", "friendly_fire": False})  
# {"hp": 0, "nickname": "", "friendly_fire": False}  
  
boss_settings({"hp": None, "debug": 42})  
# {"hp": 100, "nickname": "Гоблин", "friendly_fire": True}
```

Проверьте ещё. Пустой словарь; все три настройки равны `None`.

Объясните. Чем проверка на `None` отличается от проверки истинности? Почему `or` не подходит для этого контракта?

Портал открылся вчера

int, divmod, отрицательные числа | Слайды лекции: 10

Таймер портала показывает секунды относительно начала дня 0. Иногда портал открывается в прошлом. Маги требуют нормальные часы, а не время вида 0 дней, 0 часов, 0 минут, минус 1 секунда.

Что сделать

1. Напишите `portal_clock(seconds: int) -> tuple[int, int, int, int]`. Результат: день, час, минута, секунда.
2. День может быть отрицательным. Часы должны лежать в 0..23, минуты и секунды в 0..59.
3. Обратное преобразование должно давать исходное число: $\text{day} * 86400 + \text{hour} * 3600 + \text{minute} * 60 + \text{second}$.
4. Вход - целое число любого знака, включая очень большое. bool не передают. Не используйте float, datetime и деление /.

Примеры и ожидаемое поведение

```
portal_clock(90061)    # (1, 1, 1, 1)
portal_clock(-1)       # (-1, 23, 59, 59)
portal_clock(0)        # (0, 0, 0, 0)
portal_clock(-86400)   # (-1, 0, 0, 0)
```

Проверьте ещё. $10^{**}100$ и $-(10^{**}100)$: проверьте диапазоны компонентов и обратное преобразование.

Объясните. Почему целочисленное деление вниз даёт нужный день для отрицательного времени?

Посадка на астероид 0.3

float, относительный и абсолютный допуск | Слайды лекции: 11-13

Автопилот вычислил координату $0.1 + 0.2$. Диспетчер сравнил её с 0.3 через `==` и объявил, что корабль промахнулся. Экипаж просит более содержательное определение попадания.

Что сделать

1. Напишите `landing_hits(readings: list[float], target: float) -> list[int]`: индексы подходящих измерений в исходном порядке, начиная с нуля.
2. Попадание определяется условием: `abs(value - target) <= max(1e-9 * max(abs(value), abs(target)), 1e-12)`. Можно использовать `math.isclose` с этими допусками.
3. Все числа конечные, NaN и бесконечности отсутствуют. Вход не меняйте. Предварительно округлять числа или превращать их в строки нельзя.

Примеры и ожидаемое поведение

```
landing_hits([0.1 + 0.2, 0.3001, 0.3], 0.3)
# [0, 2]

landing_hits([1e-13, 1e-6], 0.0)
# [0]

landing_hits([1e9 + 0.5, 1e9 + 2.0], 1e9)
# [0]
```

Проверьте ещё. Пустой список; отрицательная цель; измерение, точно равное цели.

Объясните. Зачем возле нуля нужен абсолютный допуск? Почему здесь не стоит писать `round(value, 1) == round(target, 1)`?

Йожик и его двойник

str, Unicode, нормализация, уникальность | Слайды лекции: 16, 23, 27

В чате гильдии два одинаково выглядящих Йожика. Один набрал букву Й одним символом, другой собрал её из И и кратки. А утка зарегистрировалась ещё три раза, меняя регистр и пробелы.

Что сделать

1. Напишите `unique_nicknames(names: list[str]) -> list[str]`. Для каждого имени последовательно выполните `strip()`, `casefold()` и нормализацию NFC.
2. Удалите пустые результаты и повторения. Верните нормализованные имена в порядке их первого появления.
3. Исходный список не меняйте. Разные алфавиты не объединяйте: латинская а и кириллическая а остаются разными именами.

Примеры и ожидаемое поведение

```
names = [" УТКА ", "утка", "И\u0306ожик", "ЙОЖИК", " "]
unique_nicknames(names)
# ["утка", "йожик"]

unique_nicknames(["Straße", "STRASSE"])
# ["strasse"]
```

Проверьте ещё. Пустой вход; только пробелы; пара строк `é` и `е\u0301`.

Объясните. Почему одинаковое отображение не гарантирует одинаковую последовательность кодовых точек? Чем `casefold` отличается по назначению от `lower`?

Телеграмма с Марса

str и bytes, UTF-8, двоичный формат | Слайды лекции: 16, 18

Марсианский протокол предельно прост: сначала длина, потом сообщение. Но земной программист написал в заголовок число букв. Пока передавали hello, всё работало. На слове кот связь закончилась.

Что сделать

1. Напишите `pack_message(text: str) -> bytes` и `unpack_message(packet: bytes) -> str`.
2. Формат: первые 2 байта - длина полезной нагрузки в байтах, без знака, big-endian. Остаток - строка, закодированная в UTF-8. Заголовок в длину не входит.
3. При упаковке длина больше 65535 байт вызывает `ValueError`. Пустая строка допустима.
4. При распаковке неполный заголовок или несовпадение длины вызывает `ValueError`. Лишние байты тоже ошибка. Некорректный UTF-8 должен вызывать `UnicodeDecodeError`; подавлять его не нужно.

Примеры и ожидаемое поведение

```
pack_message("кот").hex()  
# "0006d0bad0bed182"  
  
unpack_message(bytes.fromhex("0006d0bad0bed182"))  
# "кот"  
  
pack_message("")  
# b"\x00\x00"
```

Проверьте ещё. Пакет из одного байта; заголовок 2 и один байт данных; 65535 и 65536 латинских букв а.

Объясните. В какой момент известна правильная длина пакета? Почему `len(text)` и `len(text.encode('utf-8'))` могут различаться?

Сапёр с общими строками

Вложенные списки и повторение ссылок | Слайды лекции: 19-22

Разработчик сапёра сэкономил одну строку кода. Теперь одна мина взрывает сразу весь столбец. Найдите место, где экономия превратилась в оружие массового поражения.

Исходный фрагмент

```
field_map = [["."] * 3] * 2
field_map[0][1] = "*"
```

Что сделать

1. Объясните, почему сломанный фрагмент меняет обе строки.
2. Напишите `minefield(rows, cols, mines) -> list[list[str]]. rows` и `cols` - неотрицательные целые; `mines` - список пар `(row, col)`, индексация с нуля.
3. Пустая клетка обозначается точкой, мина звёздочкой. Координаты гарантированно допустимы; повтор координаты ничего не меняет.
4. Каждая строка результата должна быть отдельным списком, включая пустые строки. Входной список координат не изменяйте.

Примеры и ожидаемое поведение

```
minefield(2, 3, [(0, 1)])
# [".", "*", "."], [".", ".", "."]

minefield(0, 3, []) # []
minefield(2, 0, []) # [], []
```

Проверьте ещё. Две одинаковые координаты; ручное изменение одной строки результата после вызова.

Объясните. Почему внутреннее умножение `["."] * cols` допустимо, а внешнее умножение готовой строки создаёт проблему?

Битва мемов: утка против кота

dict, tuple, стабильная сортировка | Слайды лекции: 19-20, 23-24

Гильдия выбирает мем недели. Побеждает число голосов, а при равенстве выше стоит мем, за который проголосовали раньше. Алфавитный порядок не считается демократией.

Что сделать

1. Напишите `meme_rating(votes: list[str]) -> list[tuple[str, int]]`: каждый мем и число его голосов.
2. Сортируйте по убыванию числа голосов. При равенстве сохраняйте порядок первого появления мема во входе.
3. Строки сравниваются буквально, регистр значим. Исходный список не меняйте. Используйте словарь; не вызывайте `votes.count()` для каждого мема.
4. Цель по времени: $O(n + k \log k)$, где n - число голосов, k - число разных мемов, при обычной средней стоимости операций словаря.

Примеры и ожидаемое поведение

```
meme_rating(["утка", "кот", "утка", "гусь", "кот"])
# [("утка", 2), ("кот", 2), ("гусь", 1)]

meme_rating(["z", "a"])
# [("z", 1), ("a", 1)]

meme_rating([]) # []
```

Проверьте ещё. Все голоса за один мем; все мемы разные; два названия с разным регистром.

Объясните. Какие две гарантии позволяют сохранить порядок при равном числе голосов?

Читер с тремя паспортами

Равенство, хеширование, составные ключи | Слайды лекции: 25-26, 37

Античит записал трёх игроков: True, 1 и 1.0. В базе осталась одна запись, и теперь дракон утверждает, что он утка. Для этого сервиса тип идентификатора является частью его смысла.

Исходный фрагмент

```
passports = {True: "утка", 1: "гусь", 1.0: "дракон"}  
print(len(passports))  
print(passports[True])
```

Что сделать

1. Предскажите две строки вывода сломанного примера и объясните результат.
2. Напишите `typed_counts(values) -> dict`: количество каждого значения с учётом его точного типа. Ключ результата должен быть парой `(type(value), value)`.
3. Во входном списке только встроенные `bool`, `int`, `float` и `str`, без подклассов. `float` конечные; `NaN` отсутствует. Значения одного типа сравниваются обычным `==`, поэтому `0.0` и `-0.0` считаются одним значением.
4. Вход не изменяйте. Пустой список даёт пустой словарь.

Примеры и ожидаемое поведение

```
result = typed_counts([True, 1, 1.0, "1", False, 0, True])  
assert len(result) == 6  
assert result[(bool, True)] == 2  
assert result[(int, 1)] == 1  
assert result[(float, 1.0)] == 1  
assert result[(str, "1")] == 1
```

Проверьте ещё. `False` и `0`; повтор одной строки; `0.0` и `-0.0`.

Объясните. Почему одного совпадения хешей недостаточно, чтобы любые два ключа считались одинаковыми?

В рейд опять не взяли барда

set, frozenset, операции множеств | Слайды лекции: 25, 27, 29

Две команды объединились для рейда. Маг есть у обеих, танк у одной, лекарь у другой. Бард снова узнал о сборе последним. Нужен отчёт по представленным ролям.

Что сделать

1. Напишите `raid_audit(alpha, beta, required)`. Все три аргумента - множества строк с названиями ролей.
2. Верните кортеж из трёх `frozenset`: роли обеих команд; роли ровно одной команды; обязательные роли, которых нет ни в одной команде.
3. Входные множества не изменяйте. Порядок элементов множества не важен. Весь результат должен подходить для использования ключом словаря.

Примеры и ожидаемое поведение

```
result = raid_audit(
    {"танк", "маг"},
    {"маг", "лекарь"},
    {"танк", "лекарь", "бард"},
)
assert result == (
    frozenset({"маг"}),
    frozenset({"танк", "лекарь"}),
    frozenset({"бард"}),
)
cache = {result: "состав проверен"}
```

Проверьте ещё. Пустые команды; одинаковые команды; все обязательные роли уже есть.

Объясните. Чем симметрическая разность отличается от объединения? Почему `tuple` из обычных `set` не стал бы хешируемым?

Генератор устал и ушёл

Iterable, iterator, один проход | Слайды лекции: 30-31

Датчик батарейки отдаёт измерения по одному. Аналитик сначала вычислил сумму, потом решил посчитать элементы. Датчик сообщил: 'Я уже всё рассказал'.

Исходный фрагмент

```
values = iter([3, -1, 4])
total = sum(values)
count = len(list(values)) # Почему здесь не 3?
```

Что сделать

1. Напишите `energy_report(values: Iterable[int]) -> tuple[int, int, float | None]`: количество, сумма, среднее.
2. Источник конечный и может быть одноразовым генератором. Сделайте ровно один проход, не вызывайте `len(values)`, не сохраняйте поток в список или другой контейнер.
3. Для пустого источника верните `(0, 0, None)`. Среднее непустого потока - `float`.
Дополнительная память должна содержать лишь постоянное число накопителей.
4. Измерения могут быть отрицательными. Для задачи гарантировано, что среднее представимо обычным `float`.

Примеры и ожидаемое поведение

```
stream = iter([3, -1, 4])
energy_report(stream) # (3, 6, 2.0)
list(stream)          # []

energy_report(iter([])) # (0, 0, None)
energy_report(x for x in [0, 0]) # (2, 0, 0.0)
```

Проверьте ещё. Один элемент; отрицательная сумма; источник, запрещающий второй вызов `__iter__`.

Объясните. Чем повторный обход списка отличается от повторного обхода уже исчерпанного итератора?

Лицензия на боевой клич

Класс как объект, type, метакласс | Слайды лекции: 38-42

Профсоюз боссов запрещает выходить на арену без собственного боевого клича. Проверку нужно проводить при создании класса: ждать появления первого босса уже поздно.

Исходный фрагмент

```
class CryMeta(type):
    def __new__(mcls, name, bases, namespace, **kwargs):
        # Вставьте проверку перед созданием класса.
        return super().__new__(mcls, name, bases, namespace, **kwargs)

class Boss(metaclass=CryMeta):
    abstract = True
```

Что сделать

1. Допишите CryMeta. Класс с `abstract = True` в собственном теле освобождается от проверки. Значение флага всегда `bool`, если флаг указан.
2. Все остальные классы обязаны объявить `battle_cry` в своём теле. Значение - строка с хотя бы одним непробельным символом. При нарушении поднимайте `TypeError`.
3. Унаследованные `abstract` и `battle_cry` не учитываются как собственные объявления. Создайте корректный класс `Dragon(Boss)`.

Примеры и ожидаемое поведение

```
class Dragon(Boss):
    battle_cry = "Верните мои дедлайны!"

class Silent(Dragon):
    pass # TypeError возникает уже при выполнении class.

assert type(Dragon) is CryMeta
assert type(Dragon()) is Dragon
```

Проверьте ещё. Клич из пробелов; клич-число; новый класс с собственным `abstract = True`. Ошибочные объявления запускайте отдельно или внутри `try/except`.

Объясните. Почему проверка в `__init__` экземпляра была бы слишком поздней? Какие атрибуты лежат в `namespace` создаваемого класса?